

# 04d — gRPC

## Key Concepts

| Term                        | Definition                                                                           |
|-----------------------------|--------------------------------------------------------------------------------------|
| gRPC                        | Google Remote Procedure Call — high-performance framework for calling remote methods |
| Protocol Buffers (Protobuf) | Binary serialization format used by gRPC — faster and smaller than JSON              |
| .proto file                 | Mandatory contract defining services, methods, and message structures                |
| HTTP/2                      | Transport protocol gRPC runs on — enables multiplexing and streaming                 |
| Multiplexing                | Multiple requests over one connection simultaneously                                 |
| Streaming                   | Continuous flow of messages between client and server                                |

## gRPC vs REST

|                 | REST               | gRPC                   |
|-----------------|--------------------|------------------------|
| Style           | Resource URLs      | Method/function calls  |
| Data format     | JSON (text)        | Protobuf (binary)      |
| Contract        | Optional (OpenAPI) | Mandatory (.proto)     |
| HTTP version    | HTTP/1.1           | HTTP/2                 |
| Speed           | Slower             | 5-10x faster           |
| Streaming       | No                 | Yes (4 types)          |
| Browser support | Full               | Limited (needs proxy)  |
| Best for        | Public APIs        | Internal microservices |

## .proto File Example

```
service UserService {
  rpc GetUser (UserRequest) returns (UserResponse);
}

message UserRequest {
  int32 user_id = 1;
}

message UserResponse {
```

```
int32 user_id = 1;
string name = 2;
string email = 3;
}
```

Auto-generates client and server code in any language from this single file.

---

## Why gRPC is Fast

1. **Protobuf** — binary format, 3-10x smaller than JSON, faster to parse
  2. **HTTP/2** — multiplexing (multiple calls on one connection), header compression
  3. **Code generation** — no runtime reflection, strongly typed
- 

## 4 Communication Types

| Type                    | Pattern                         | Use Case                      |
|-------------------------|---------------------------------|-------------------------------|
| Unary                   | 1 request → 1 response          | Standard call like REST       |
| Server streaming        | 1 request → stream of responses | Live feed, large download     |
| Client streaming        | Stream of requests → 1 response | File upload, batch send       |
| Bidirectional streaming | Stream both ways                | Chat, real-time collaboration |

---

## When to Use

### Use gRPC:

- Internal microservice-to-microservice communication
- Performance-critical paths
- Need streaming
- Polyglot environment (services in different languages)

### Use REST:

- Public-facing APIs (browsers can't call gRPC directly)
  - Simplicity over performance
  - Third-party developer consumption
- 

## Industry Examples

| Company | Usage                                       |
|---------|---------------------------------------------|
| Google  | Internal service-to-service communication   |
| Netflix | Microservices communication for performance |

---

| Company      | Usage                                                                    |
|--------------|--------------------------------------------------------------------------|
| Uber         | Migrated internal APIs from REST to gRPC — reduced payload significantly |
| ASP.NET Core | First-class gRPC support since .NET 3.0                                  |

## Interview Questions

**Q: What is gRPC and how does it differ from REST?** A: gRPC calls remote methods directly instead of HTTP URLs. It uses binary Protobuf instead of JSON, runs on HTTP/2, and requires a .proto contract. It's 5-10x faster than REST.

**Q: What are Protocol Buffers?** A: Binary serialization format — smaller and faster than JSON. Defined in .proto files which auto-generate client/server code in any language.

**Q: When would you choose gRPC over REST?** A: Internal microservices where performance matters, polyglot environments, or when you need streaming. REST for public APIs since browsers can't call gRPC directly.

**Q: What is bidirectional streaming in gRPC?** A: Both client and server send a stream of messages simultaneously over one connection — used for real-time features like chat or live collaboration.